

Analyse des algorithmes de tri

Conception Logicielle 3

Thomas Matthieu Marie Vianney Siaghi Massinissa Tellier Basile

2 avril 2026

Table des matières

- 1 Contexte et objectifs
- 2 Génération des données
- 3 Algorithmes de tri
- 4 Expérimentation
- 5 Analyse des résultats
- 6 Améliorations possibles
- 7 Conclusion
- 8 Questions

Question scientifique

Comment les paramètres d'entrée d'un tableau (taille, degré et zone de désordre) influencent-ils les performances des algorithmes de tri ?

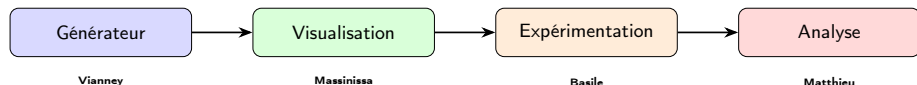
Métriques mesurées

- Comparaisons
- Accès mémoire
- Échanges
- Temps d'exécution

Démarche

- 1 Modélisation
- 2 Génération
- 3 Expérimentation
- 4 Analyse

Architecture du projet



Outils utilisés

- Java (Swing) : visualisation
- Bash : automatisation des expériences
- Python (matplotlib) : tracé des courbes

Paramètres expérimentaux

- 5 algorithmes de tri
- Tailles variées
- 4 modes de désordre
- Pourcentages de 10 % à 100 %

Définition

Un tableau est dit **partiellement désordonné** si une fraction de ses éléments est mélangée selon une zone définie.

Paramètre 1 : pourcentage de désordre

De 10 % à 100 % par pas de 10 %.

Paramètre 2 : zone de désordre

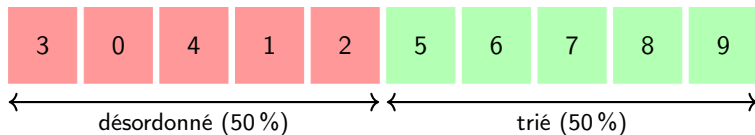
- Mode 1 — positions **aléatoires** dans tout le tableau
- Mode 2 — **début** du tableau
- Mode 3 — **milieu** du tableau
- Mode 4 — **fin** du tableau

Génération : Exemple de sortie

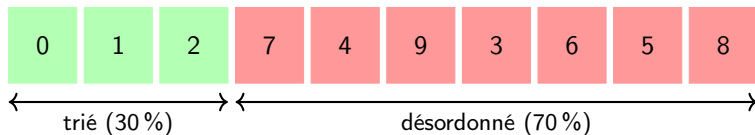
```
0 1 2 3 4 5 6 7 8 9
désordre aleatoire (1) :
1 3 2 8 4 0 6 7 5 9
désordre début (2) :
3 4 1 0 2 5 6 7 8 9
désordre milieu (3) :
0 1 2 5 7 4 3 6 8 9
désordre fin (4) :
0 1 2 3 4 8 9 6 7 5
```

Génération : Illustration

Exemple 1 : 50 % de désordre en début (Mode 2)



Exemple 2 : 70 % de désordre en fin (Mode 4)

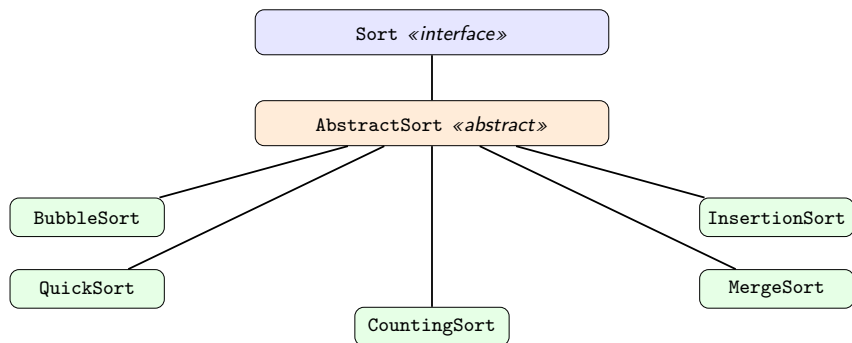


Génération : Classe Generator

```
1  generateurTab(taille , pourcentageAleatoire ,desordre) {
2      tab <- [0, 1, ..., taille-1]
3      n <- taille * pourcentage / 100
4      si desordre == 1:
5          choisir n indices aleatoires
6          extraire leurs valeurs
7          melanger (shuffle)
8          remplacer aux memes indices
9
10     sinon si desordre == 2:
11         prendre les n premiers elements
12         melanger (shuffle)
13         remettre au debut
14
15     sinon si desordre == 3:
16         prendre n elements au milieu
17         melanger (shuffle)
18         remettre au milieu
19
20     sinon si desordre == 4:
21         prendre les n derniers elements
22         melanger (shuffle)
23         remettre a la fin
24     retourner tab
25 }
```



Architecture des algorithmes



Architecture des algorithmes - Conception

Patterns utilisés

- **Template Method** : `sort()` finale, `sortImpl()` abstraite
- **Strategy** : algorithme substituable à l'exécution
- **Observer** : modèle notifie la vue

Extensibilité

Ajouter un algorithme = créer une classe qui étend `AbstractSort` et implémenter `sortImpl()`.

Aucune modification du code existant.

Instrumentation : AbstractSort

Chaque opération est une sonde :

- `read()` : incrémente les accès
- `write()` : incrémente les accès
- `isLess()` : incrémente les comparaisons
- `swap()` : incrémente les échanges

Chaque sonde :

- 1 Vérifie l'interruption du thread
- 2 Met à jour le chronomètre
- 3 Notifie le modèle de visualisation
- 4 Incrémente le compteur

Template Method

```
public final void sort(int[] a) {  
    resetMetrics();  
    startTime = System.nanoTime();  
    sortImpl(a); // surchargee  
    timeNano = System.nanoTime() - startTime;  
}  
  
protected abstract void sortImpl(int[] array);
```

Annulation propre

`cancel(true)` sur le worker → interruption propagée → les sondes deviennent des no-ops

Les cinq algorithmes

Algorithme	Moy.	Pire	En place	Stable
Bubble Sort	$O(n^2)$	$O(n^2)$	Oui	Oui
Insertion Sort	$O(n^2)$	$O(n^2)$	Oui	Oui
Quick Sort	$O(n \log n)$	$O(n^2)$	Oui	Non
Merge Sort	$O(n \log n)$	$O(n \log n)$	Non	Oui
Counting Sort	$O(n + \max)$	$O(n + \max)$	Non	Oui

Particularités notables

- **Bubble Sort** : arrêt anticipé si aucun échange détecté
- **Quick Sort** : pivot aléatoire pour réduire le risque du pire cas
- **Counting Sort** : non comparatif, réservé aux entiers positifs, insensible au désordre

Visualisation interactive

Interface Swing - MVC

- `ArrayBarChart` : barres colorées
- `MetricsDisplay` : métriques temps réel
- `ControlPanel` : démarrer, pause, reset, nouveau tableau, vitesse

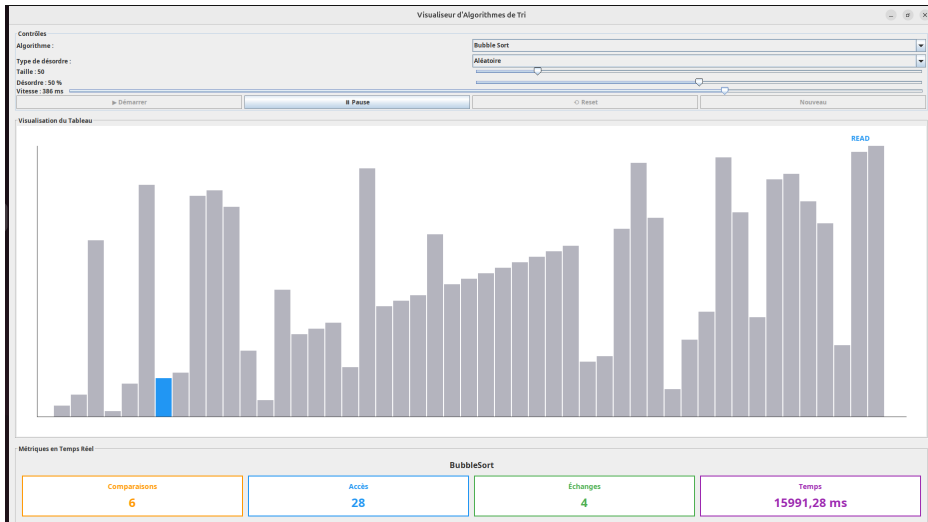
Code couleur des opérations

- **Orange** : comparaison
- **Vert** : échange
- **Bleu** : lecture
- **Cyan** : écriture

Contrôle du tri

- Tri exécuté dans un `SwingWorker` (thread séparé, EDT non bloqué)
- Pause : boucle d'attente dans `updateVisualization()`
- Reset : `cancel(true)` + remise à zéro de l'état pause
- Vitesse : 10 ms à 500 ms par opération

Visualisation interactive : Démonstration



Objectif

Collecter de nombreuses métriques

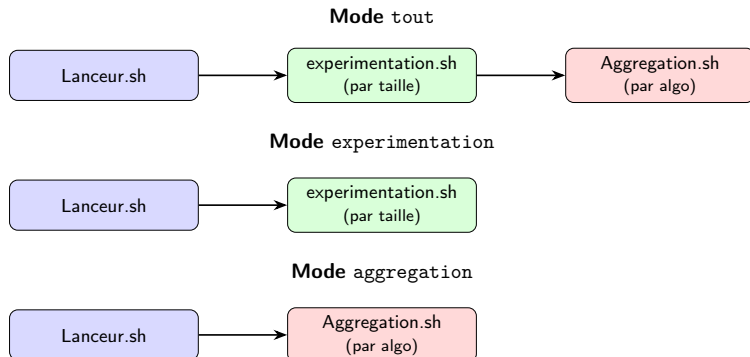
Paramètres balayés

- **Tailles** 100, ..., 100 000
- **Désordre** : 0 %, 10 %, ..., 100 %
- **Modes** : 1, 2, 3, 4
- **Algorithmes** : 5
- **Répétitions** : 100 par configuration

Collecte des données

- **Parallélisation**
- Automatisation par script Bash
- Appel de `Experimentation.java`
- Résultats stockés dans des fichiers CSV

Expérimentation : Architecture du script Bash



Séparation experimentation / agrégation : permet de relancer l'agrégation sans ré-exécuter les tris.

Expérimentation : Flux d'une exécution

Pour chaque triplet (taille, désordre, type), experimentation.sh :

- 1 Appelle Experimentation.java → 100 répétitions par algorithme
- 2 Filtre les lignes préfixées [csv] sur la sortie standard
- 3 Placement dans un fichier temporaire du résultat
- 4 Injecte taille;désordre;type en tête de chaque ligne
- 5 Append dans size_\${taille}.csv

Export Java → Bash

```
grep '^\[csv\]' "$sortie" | while read -r line; do
    data=$(echo "$line" | cut -d ' ' -f 2-)
    echo "${size};${disorder};${type};${data}" >> "$fichier"
done
```

Expérimentation : Résultats par taille

	A	B	C	D	E	F	G	H
1	taille	desordre	typeDesordre	algo	accesses	comparaisons	swaps	timeNano
2	100	0		1 CountingSort	301	0	0	20720
3	100	0		1 QuickSort	2495	974	442	187410
4	100	0		1 InsertionSort	297	99	0	30620
5	100	0		1 BubbleSort	198	99	0	20800
6	100	0		1 MergeSort	1344	356	0	100580
7	100	0		1 CountingSort	301	0	0	16270
8	100	0		1 QuickSort	2595	969	476	122980
9	100	0		1 InsertionSort	297	99	0	11150
10	100	0		1 BubbleSort	198	99	0	9720
11	100	0		1 MergeSort	1344	356	0	147349

Expérimentation : Résultats obtenus

Volume des données

- 9 fichiers size_*.csv
- 5 fichiers par algorithme
- Export automatique via balises [csv]

Validation automatique

- Vérification du tri (isSorted)

Prêt pour l'analyse

Données structurées
Python + matplotlib

Expérimentation : Résultats post-aggrégation

	A	B	C	D	E	F	G	H
1	taille	desordre	typeDesordre	algo	acces	comparaisons	echanges	temps
2	###	0		1 QuickSort	6734976	3007620	1175672	22260498
3	###	0		1 QuickSort	6621703	2958520	1154286	7683340
4	###	0		1 QuickSort	7184938	3223600	1253957	7459100
5	###	0		1 QuickSort	6314253	2905470	1069651	5757927
6	###	0		1 QuickSort	6521697	2980221	1113746	7439960
7	###	0		1 QuickSort	6336236	2897153	1079674	4615806
8	###	0		1 QuickSort	6761320	3083293	1159394	6997329
9	###	0		1 QuickSort	6608247	3037092	1123635	4288546
10	###	0		1 QuickSort	6823934	3096968	1175738	4419076
11	###	0		1 QuickSort	6451181	2982800	1089430	5725957
12	###	0		1 QuickSort	6512460	2984034	1109482	4378956
13	###	0		1 QuickSort	6589577	2966600	1140954	3930275
14	###	0		1 QuickSort	6655950	3011877	1148167	6596019
15	###	0		1 QuickSort	6836364	3130554	1168586	4837767
16	###	0		1 QuickSort	6769193	3015071	1184740	6744499
17	###	0		1 QuickSort	6678298	3035653	1147534	4734566
18	###	0		1 QuickSort	6627269	2986940	1146733	5039087
19	###	0		1 QuickSort	6950657	3145859	1201654	7898370
20	###	0		1 QuickSort	6683234	2985572	1165915	4966886

Analyse : Lecture des données

- On utilise les fichier CSV par algorithme, chaque ligne représente une exécution
- Les mesures sont regroupés par clé (algo, taille, désordre, type) et accumulées dans des liste pour gérer les répétitions
- une moyenne est calculée pour chaque combinaisons afin de lisser les mesures

```
1 key = (algo_name, size, disorder, dtype)
2 data[key]["compare"].append(compare)
3 # ...
4 mean_data[key] = {"compare": np.mean(metrics["compare"]), ...}
```

Listing 2 – Analyse du CSV

Analyse : Génération des courbes

- **Comparaison inter-algorithmes** : pour chaque combinaison (taille, type, métrique) - une courbe par algo en fonction du désordre
→ Qui est le plus efficace ?
- **Influence de la localisation** : pour chaque combinaison (algo, taille, métrique) - une courbe par type de désordre
→ Est-ce que la répartition change quelque chose ?
- Une version **filtrée** sans BubbleSort et InsertionSort a été générée pour révéler le comportement des algorithmes efficaces, écrasés visuellement par les $O(n^2)$

Résultat : Comparaisons

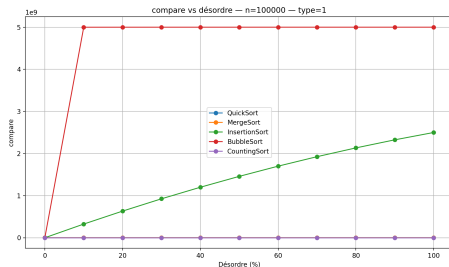


Figure – Tout algorithme

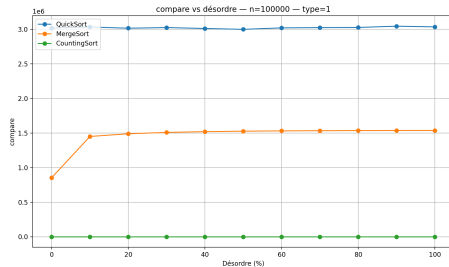


Figure – Sans $O(n^2)$

Résultat : Accès mémoire & Échanges

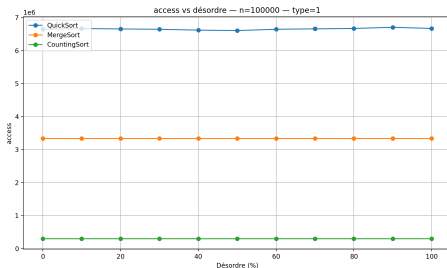


Figure – Accès mémoire

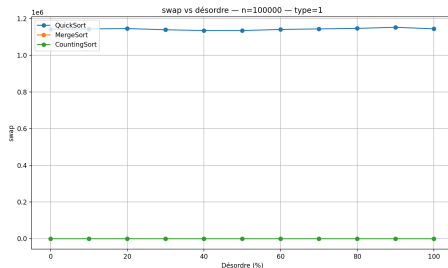


Figure – Échanges

Résultat : Temps d'exécution & Influence du désordre

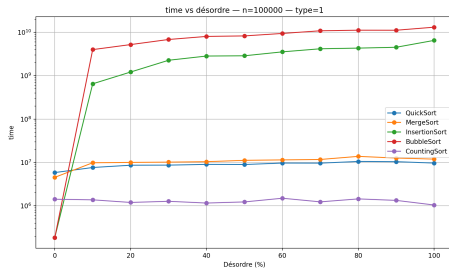


Figure – Temps (échelle logarithmique)

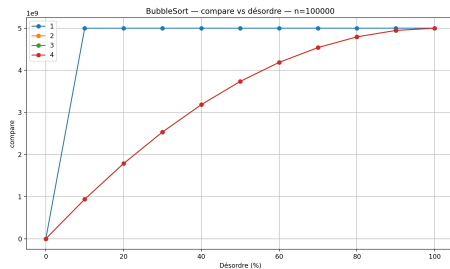


Figure – Influence du désordre

Résultat : Synthèse

Algo	Complexité	Sensibilité
BubbleSort	$O(n^2)$	Forte
InsertionSort	$O(n^2)$	Modérée
MergeSort	$O(n \log n)$	Modérée
QuickSort	$O(n \log n)$	Faible
CountingSort	$O(n+k)$	Nulle

Améliorations possibles

Algorithmes

- HeapSort, ShellSort, TimSort
- Étendre AbstractSort suffit : aucune modification existante

Visualisation

- Comparaison côte-à-côte de deux algorithmes en temps réel

Expérimentation

- Pouvoir modifier simplement la taille des tableaux
- Mettre en pause et reprendre le cycle d'expérience
- Reprendre le cycle en cas d'interruption
- Optimiser la phase d'expérimentation

Architecture

- Tests unitaires sur les algorithmes

Conclusion

Ce qui a été réalisé

- Architecture MVC complète en Swing
- 5 algorithmes visualisables
- Génération de tableaux avec 4 types de désordre
- Expérience lancée avec Bash et résultats stockées en CSV
- Analyse des résultats avec Python

Comment les paramètres d'entrée d'un tableau influencent-ils les performances des algorithmes de tri ?

- Impact de la **taille**
- Impact du **pourcentage** de désordre
- Impact de la **zone** de désordre
- Comparaison $O(n^2)$ vs $O(n \log n)$

Merci pour votre attention.

Avez vous des questions ?

Sources

- Sorting Algorithms : GeeksForGeeks
<https://www.geeksforgeeks.org/dsa/sorting-algorithms/>
- 15 Sorting Algorithms in 6 Minutes : YouTube
<https://www.youtube.com/watch?v=kPRA0W1kECg>
- Introduction to Algorithms : Cormen et al., MIT Press
- Documentation Java : <https://docs.oracle.com/en/java/>
- Lancement de java sans compilation <https://www.infoq.com/fr/articles/single-file-execution-java11/>
- Prendre des arguments de commande en Java
https://youtu.be/5Ju8n2r1Eyw?si=xWz4Y8v_ygWx16G9